

Private Network Service Platform

Private DNS, HTTPS/TLS Termination, Reverse Proxy, Load Balancing, HTTP Caching and Packet-Level Analysis on a Two-Mac Network

Team member	Roll number	Machine and responsibilities
Mansi Agarwal	2401010264	Mac 1 (10.7.21.182): private DNS (dnsmasq), Backend A, test client
Tanisha Dhiman	2401010475	Mac 2 (10.7.21.206): nginx edge (HTTPS), Backend B, DNS client

Course	Computer Networks (Phase 1)
Institution	Newton School of Technology, Rishihood University
Infrastructure	Two physical MacBooks on one Wi-Fi LAN (no virtual machines)
Private domain	app.team1.test (and api.team1.test), HTTPS on port 8443
Submission	October 2026

Table of Contents

1. Abstract	3
2. Introduction and Objectives	3
3. Requirements-to-Evidence Matrix	4
4. System Architecture and Network Topology	5
5. Private DNS and Network Connectivity (A1-A5)	6
5.1 A1 - Machine addresses and roles	6
5.2 A2 - Private DNS with dnsmasq	7
5.3 A3 - Cross-machine DNS (documented limitation)	7
5.4 A4 - Public DNS does not know the private domain	8
5.5 A5 - Connectivity in both directions	8
6. Backends, nginx Reverse Proxy and HTTPS (B1, B3)	8
6.1 Backend services	8
6.2 B3 - nginx configuration	9
6.3 B1 - HTTPS with a verified certificate	9
7. Load Balancing (B2)	10
8. Wireshark Network Analysis (C1-C3)	12
8.1 C1 - DNS query and response	12
8.2 C2 - TCP three-way handshake	12
8.3 C3 - TLS handshake and certificate inspection	13
9. HTTP Caching (D1, D2)	15
9.1 D1 - Cache headers and conditional request	15
9.2 D2 - Explanation	16
10. Failure and Recovery Demonstration (D3)	16
11. Testing and Validation Summary	18
12. Security Considerations	18
13. Challenges and Observations	18
14. Limitations	19
15. Conclusion	19
Appendix A - nginx Configuration	20
Appendix B - Verification Commands	20
Appendix C - Evidence Index	21

1. Abstract

This report documents the design, implementation and verification of a **private network service platform** built on two physical MacBooks connected to the same Wi-Fi network. Mac 1 hosts a private DNS server (dnsmasq), Backend A and the test client. Mac 2 hosts an nginx edge server that terminates HTTPS on port 8443 and load-balances requests between Backend A (Mac 1, port 3001) and Backend B (Mac 2, port 3002). All services are reached through the private domain **app.team1.test**, which resolves to the nginx edge (10.7.21.206) and is deliberately invisible to public DNS.

Every layer of the stack was exercised and evidenced: DNS resolution and its public-DNS negative control, ICMP connectivity in both directions, a verified TLS 1.3 session using a certificate whose name matches the domain (no `curl -k` anywhere), round-robin load balancing, HTTP caching with *Cache-Control*, *ETag* and *304 Not Modified*, and a backend-failure scenario in which service continued uninterrupted through Backend B and normal two-backend operation resumed after Backend A was restarted. Wireshark captures at the DNS, TCP and TLS layers show what happened on the wire.

One requirement could not be fully met, and this report says so plainly: **cross-machine DNS (A3)**. The university-managed firewall blocked Mac 2 from reaching dnsmasq on port 53 of Mac 1. Local resolution on Mac 1 was verified successfully, and the limitation is documented in Sections 5.3 and 14 rather than presented as a success.

Result at a glance	Evidence
app.team1.test resolves to 10.7.21.206 (private DNS); public resolver 8.8.8.8 returns NXDOMAIN	Figures 3-5
0% packet loss between the Macs in both directions (4/4 echo replies each way)	Figures 6-7
HTTPS on :8443, certificate verified without -k, TLS 1.3, HTTP 200	Figure 9
Six requests distributed A, B, A, B, A, B across the two backends	Figure 10
DNS, TCP three-way handshake and TLS handshake captured in Wireshark	Figures 11-16
Cache-Control max-age=60 + ETag; conditional request returns 304 Not Modified	Figures 17-18
Backend A stopped: 6/6 requests still HTTP 200 via Backend B; A/B alternation restored after restart	Figures 19-22

2. Introduction and Objectives

A web service that is reachable by name, protected by TLS, spread over several servers and observable on the wire combines five independent protocol layers: name resolution (DNS), transport (TCP), session security (TLS), application semantics (HTTP) and request distribution (a reverse proxy). Each is simple in isolation; the engineering challenge of this project is making them cooperate across two real machines and proving, with observable evidence, that every layer behaves as designed.

The project uses the reserved **.test** top-level domain, so no public DNS record or public certificate authority is involved. A client asks the private DNS server for the address of *app.team1.test*, opens a TCP connection to the nginx edge on port 8443, completes a TLS handshake, and sends an HTTP request that nginx forwards to one of two backends.

Objectives

ID	Objective	Section
A1	Document both machines, their addresses, the Wi-Fi interface and roles	3, 5.1
A2	Configure a private DNS server that maps the project domain to the nginx edge	5.2
A3	Resolve the domain from the second machine via the private DNS server	5.3
A4	Prove the private domain is not publicly resolvable	5.4
A5	Prove IP connectivity between the machines in both directions	5.5
B1-B3	Serve HTTPS with a valid certificate; load-balance two backends through nginx	6, 7
C1-C3	Capture and interpret DNS, TCP and TLS traffic in Wireshark	8
D1-D2	Demonstrate and explain HTTP caching (Cache-Control, ETag, 304)	9
D3	Demonstrate failure of one backend, service continuity and recovery	10

Verification principles

- **Domain names, not IP addresses**, were used in every final HTTPS demonstration.
- **Certificate verification was never disabled**: `curl -k` was not used at any point.
- HTTPS is served on **port 8443**, so every URL carries an explicit `:8443`.
- Screenshots are reproduced as captured; only blank margins were trimmed to keep terminal text legible.

3. Requirements-to-Evidence Matrix

Each requirement is mapped to the implementation that satisfies it and to the screenshot(s) stored in the repository under *evidence/*. Status values: **PASS** = demonstrated and evidenced; **LIMITATION** = partially achieved, with the constraint documented honestly.

ID	Requirement	Implementation	Evidence	Status
A1	Machine IPs, interface and roles	Mac 1 10.7.21.182, Mac 2 10.7.21.206, both Wi-Fi en0	A1-dns	PASS
A2	Private DNS configured	dnsmasq on Mac 1: address= entries for app and api names	A2-domain-resolution	PASS
A3	Cross-machine DNS (Mac 2 to Mac 1)	Blocked by university-managed firewall on port 53; local resolution on Mac 1 verified instead	A3-cross-machine-dns	LIMITATION
A4	Domain not publicly resolvable	dig @8.8.8.8 returns NXDOMAIN	A4-public-dns	PASS
A5	Bidirectional connectivity	ping -c 4 each way: 0% loss	A5-connectivity	PASS
B1	HTTPS with verified certificate	nginx TLS on :8443; curl -v without -k: verify ok, HTTP 200	B1-https	PASS
B2	Load balancing across A and B	nginx upstream with two servers; X-Backend alternates	B2-load-balancing	PASS
B3	nginx configuration	Upstream group, listen 8443 ssl, proxy_pass	B3-nginx	PASS
C1	DNS packets in Wireshark	Query and response on lo0 (Mac 1), A 10.7.21.206	C1-dns-wireshark	PASS
C2	TCP three-way handshake	SYN, SYN-ACK, ACK to 10.7.21.206:8443 (Mac 2 capture)	C2-tcp-tls	PASS

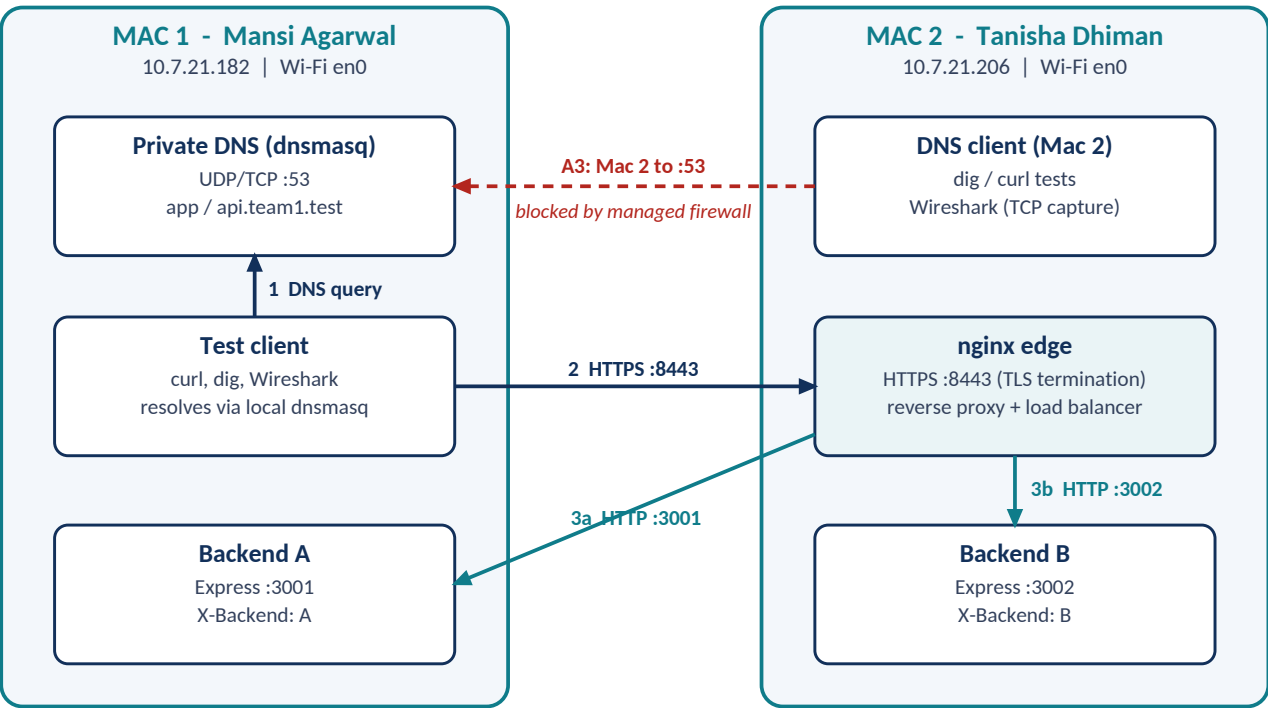
ID	Requirement	Implementation	Evidence	Status
C3	TLS handshake and certificate	ClientHello, ServerHello, Certificate (11), Application Data	C3-tls-details	PASS
D1	HTTP caching headers and 304	/api/cache: max-age=60, ETag "cache-v1", If-None-Match gives 304	D1-caching	PASS
D2	Caching explanation	Written explanation (Section 9.2)	docs/D2-caching	PASS
D3	Failure, continuity and recovery	Backend A stopped, B-only 200s, nginx error log, A restored	D3-failure-recovery	PASS

Honest scoping of A3

Requirement A3 asks for `dig app.team1.test` on Mac 2 to reach the DNS server on Mac 1. That step could not be completed because the university-managed firewall blocked Mac 2 from reaching dnsmasq on port 53. The report does not claim A3 as successful. What was verified is local resolution on Mac 1 (`dig @127.0.0.1`), the dnsmasq configuration itself, and the Wireshark capture of a real DNS exchange (C1).

4. System Architecture and Network Topology

The two Macs share one Wi-Fi LAN. Mac 1 concentrates the naming service and one backend; Mac 2 is the single public-facing entry point. The client never talks to a backend directly: it resolves the name, connects to nginx, and nginx chooses a backend per request.



Solid arrows: verified request path. Dashed red: cross-machine DNS path blocked by the university firewall.

Figure 1. Architecture of the platform: DNS and Backend A on Mac 1, nginx edge and Backend B on Mac 2.

Machine	Address	Interface	Role	Services and ports
Mac 1 (Mansi)	10.7.21.182	Wi-Fi en0	Private DNS, Backend A, test client	dnsmasq :53; Backend A :3001
Mac 2 (Tanisha)	10.7.21.206	Wi-Fi en0	nginx edge, Backend B, DNS client	nginx HTTPS :8443; Backend B :3002

Component responsibilities

- **dnsmasq (Mac 1).** Listens on loopback and 10.7.21.182, answers authoritatively for *app.team1.test* and *api.team1.test* with the nginx edge address 10.7.21.206, and forwards all other names to public resolvers.
- **nginx (Mac 2).** Terminates TLS on 8443, forwards every path to the upstream group, and adds Host, X-Real-IP, X-Forwarded-For and X-Forwarded-Proto headers.
- **Backend A / Backend B.** Small Express (Node.js / TypeScript) services that return JSON and tag every response with *X-Backend: A* or *X-Backend: B*, which makes the load-balancing decision directly observable.
- **Test client (Mac 1).** Runs dig, curl and Wireshark. Because Mac 1 runs the DNS server, the domain resolves locally there.

End-to-end request flow

Step	Stage	What happens
1	DNS	Client asks dnsmasq for the A record of <i>app.team1.test</i> ; the answer is 10.7.21.206.
2	TCP	Client opens a connection to 10.7.21.206:8443 (SYN, SYN-ACK, ACK).
3	TLS	ClientHello carries the server name; nginx presents its certificate; session keys are agreed; HTTP continues inside encrypted records.
4	Proxy	nginx applies <code>proxy_pass</code> to the upstream group and sets forwarding headers.
5	Balance	nginx selects Backend A (10.7.21.182:3001) or Backend B (:3002).
6	Respond	The backend returns JSON with X-Backend; nginx relays it to the client over TLS.

5. Private DNS and Network Connectivity (A1-A5)

5.1 A1 - Machine addresses and roles

Both Macs use the Wi-Fi interface **en0**. The hardware-port listing below was captured on Mac 1 and confirms that the Wi-Fi adapter is device **en0**; Mac 1 holds 10.7.21.182 and Mac 2 holds 10.7.21.206 (Table in Section 4).

```
mansiagarwal@Mansis-MacBook-Pro-6 ~ % networksetup -listallhardwareports

Hardware Port: Ethernet Adapter (en3)
Device: en3
Ethernet Address: fa:82:06:91:75:ff

Hardware Port: Ethernet Adapter (en4)
Device: en4
Ethernet Address: fa:82:06:91:75:00

Hardware Port: Thunderbolt Bridge
Device: bridge0
Ethernet Address: 36:6d:d2:7c:56:80

Hardware Port: Wi-Fi
Device: en0
Ethernet Address: 34:66:91:ef:34:fd

Hardware Port: Thunderbolt 1
Device: en1
Ethernet Address: 36:6d:d2:7c:56:80

Hardware Port: Thunderbolt 2
Device: en2
Ethernet Address: 36:6d:d2:7c:56:84

VLAN Configurations
=====
mansiagarwal@Mansis-MacBook-Pro-6 ~ %
```

Figure 2. A1 - `networksetup -listallhardwareports` on Mac 1: the Wi-Fi hardware port is device **en0**.

5.2 A2 - Private DNS with dnsmasq

dnsmasq on Mac 1 is configured to answer on loopback and on the LAN address, and to map both project names to the nginx edge. The configuration lines actually active in the running file were extracted with *grep*:

```
=====
mansiagarwal@Mansis-MacBook-Pro-6 ~ % grep -E '^(address=|listen-address=|interface=)' /opt/homebrew/etc/dnsmasq.conf
listen-address=127.0.0.1,10.7.21.182
address=/app.team1.test/10.7.21.206
address=/api.team1.test/10.7.21.206
mansiagarwal@Mansis-MacBook-Pro-6 ~ % █
```

Figure 3. A2 - Active dnsmasq directives: listen-address and the two address= records pointing to 10.7.21.206.

```
# config/dnsmasq.conf (repository copy)
port=53
listen-address=127.0.0.1,10.7.21.182
bind-interfaces
address=/app.team1.test/10.7.21.206
address=/api.team1.test/10.7.21.206
server=1.1.1.1
server=8.8.8.8
cache-size=150
log-queries
log-facility=/tmp/team1-dns.log
```

Both names deliberately map to **10.7.21.206**, the nginx edge, not to the backends, so every client request enters through the reverse proxy. Names outside the project are forwarded to 1.1.1.1 and 8.8.8.8, and query logging is enabled for troubleshooting.

5.3 A3 - Cross-machine DNS (documented limitation)

The A3 test requires Mac 2 to run *dig app.team1.test* against the dnsmasq server on Mac 1. This **could not be completed**: the university-managed firewall blocked Mac 2 from reaching port 53 on Mac 1. What was verified instead is that dnsmasq itself works correctly on Mac 1. The query below returned status NOERROR with the authoritative flag (aa) set and the answer 10.7.21.206, from the server at 127.0.0.1#53.

```
mansiagarwal@Mansis-MacBook-Pro-6 ~ % dig @127.0.0.1 app.team1.test

; <<>> DiG 9.20.29 <<>> @127.0.0.1 app.team1.test
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 18736
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
app.team1.test.                IN      A

;; ANSWER SECTION:
app.team1.test.                0       IN      A      10.7.21.206

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Sun Oct 04 20:30:06 IST 2026
;; MSG SIZE rcvd: 59

mansiagarwal@Mansis-MacBook-Pro-6 ~ % █
```

Figure 4. A3 - *dig @127.0.0.1 app.team1.test* on Mac 1: NOERROR, A record 10.7.21.206. Cross-machine resolution was blocked by the managed firewall.

Limitation, stated plainly

A3 is **not** claimed as successful. The failure is a network-policy constraint outside the team's control, not a configuration error: the same dnsmasq server answers correctly locally and its answer is confirmed on the wire in C1. All end-to-end HTTPS demonstrations were therefore run from Mac 1, where the private domain resolves.

5.4 A4 - Public DNS does not know the private domain

From Mac 2, the same name was queried against Google Public DNS. The response is **NXDOMAIN** with a root SOA in the authority section, which proves that *app.team1.test* exists only inside the private DNS server.

```
tanisha@Tanishas-MacBook-Pro ~ % dig @8.8.8.8 app.team1.test
; <<> DiG 9.10.6 <<> @8.8.8.8 app.team1.test
; (1 server found)
; global options: +cmd
; Got answer:
; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 36752
; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
; QUESTION SECTION:
; app.team1.test.                IN      A
;
; AUTHORITY SECTION:
;                               86237  IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026100400 1800 900 604800 86400
;
; Query time: 16 msec
; SERVER: 8.8.8.8#53(8.8.8.8)
; WHEN: Sun Oct 04 20:31:35 IST 2026
; MSG SIZE rcvd: 116
tanisha@Tanishas-MacBook-Pro ~ %
```

Figure 5. A4 - dig @8.8.8.8 app.team1.test on Mac 2: status NXDOMAIN, no answer record.

5.5 A5 - Connectivity in both directions

ICMP echo tests confirm Layer-3 reachability between the Macs before any application-layer testing. Both directions show **0.0% packet loss**. Round-trip times vary noticeably (Mac 1 to Mac 2 averages 109.5 ms; Mac 2 to Mac 1 averages 126.7 ms), which is typical of a shared, managed Wi-Fi network.

```
mansiagarwal@Mansis-MacBook-Pro-6 ~ % ping -c 4 10.7.21.206
PING 10.7.21.206 (10.7.21.206): 56 data bytes
64 bytes from 10.7.21.206: icmp_seq=0 ttl=64 time=37.218 ms
64 bytes from 10.7.21.206: icmp_seq=1 ttl=64 time=291.055 ms
64 bytes from 10.7.21.206: icmp_seq=2 ttl=64 time=12.971 ms
64 bytes from 10.7.21.206: icmp_seq=3 ttl=64 time=96.631 ms

--- 10.7.21.206 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 12.971/109.469/291.055/109.168 ms
mansiagarwal@Mansis-MacBook-Pro-6 ~ %
```

Figure 6. A5 - Mac 1 to Mac 2: ping -c 4 10.7.21.206, 4/4 received, 0.0% loss.

```
tanisha@Tanishas-MacBook-Pro ~ % ping -c 4 10.7.21.182
PING 10.7.21.182 (10.7.21.182): 56 data bytes
64 bytes from 10.7.21.182: icmp_seq=0 ttl=64 time=139.369 ms
64 bytes from 10.7.21.182: icmp_seq=1 ttl=64 time=116.389 ms
64 bytes from 10.7.21.182: icmp_seq=2 ttl=64 time=150.350 ms
64 bytes from 10.7.21.182: icmp_seq=3 ttl=64 time=92.493 ms

--- 10.7.21.182 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 92.493/126.650/150.350/24.691 ms
tanisha@Tanishas-MacBook-Pro ~ %
```

Figure 7. A5 - Mac 2 to Mac 1: ping -c 4 10.7.21.182, 4/4 received, 0.0% loss.

6. Backends, nginx Reverse Proxy and HTTPS (B1, B3)

6.1 Backend services

Both backends are Express services (TypeScript) that bind to all interfaces. A small middleware stamps every response with *X-Backend*, and each backend exposes */*, */api/status* and the caching endpoint */api/cache*. Backend A listens on **:3001** (Mac 1) and Backend B on **:3002** (Mac 2).

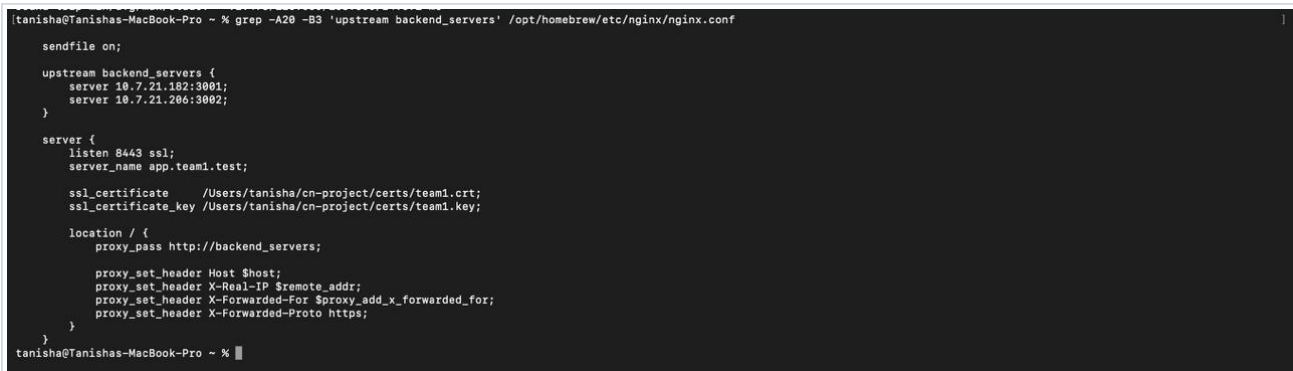
```
// config/backend-a/server.ts (excerpt)
app.use((_req, res, next) => {
  res.setHeader("X-Backend", "A");    // identifies the serving backend
  next();
});

app.get("/api/status", (_req, res) => {
  res.json({ backend: "A", status: "ok" });
});

app.get("/api/cache", (req, res) => {
  const etag = '"cache-v1"';
  if (req.headers["if-none-match"] === etag) { // client copy still valid
    res.status(304);
    res.setHeader("Cache-Control", "max-age=60");
    res.setHeader("ETag", etag);
    return res.end();
  }
  res.setHeader("Cache-Control", "max-age=60");
  res.setHeader("ETag", etag);
  res.json({ backend: "A", cached: true, message: "This response supports HTTP caching" });
});
```

6.2 B3 - nginx configuration

nginx on Mac 2 defines an upstream group with both backends and a server block that listens for TLS on **8443** for *app.team1.test*. Every request path is forwarded with *proxy_pass* to the group, so nginx acts at once as reverse proxy and load balancer. No balancing method is configured, so nginx uses its default (round-robin).



```
tanisha@Tanishas-MacBook-Pro ~ % grep -A20 -B3 'upstream backend_servers' /opt/homebrew/etc/nginx/nginx.conf
sendfile on;

upstream backend_servers {
    server 10.7.21.182:3001;
    server 10.7.21.206:3002;
}

server {
    listen 8443 ssl;
    server_name app.team1.test;

    ssl_certificate /Users/tanisha/cn-project/certs/team1.crt;
    ssl_certificate_key /Users/tanisha/cn-project/certs/team1.key;

    location / {
        proxy_pass http://backend_servers;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
tanisha@Tanishas-MacBook-Pro ~ %
```

Figure 8. B3 - nginx.conf excerpt: upstream group with Backend A (10.7.21.182:3001) and Backend B (:3002), listen 8443 ssl, proxy_pass and forwarding headers.

Backend B runs on the same machine as nginx; its upstream entry addresses it by Mac 2's own LAN address. The forwarding headers preserve the original host (*Host*), the client address (*X-Real-IP*, *X-Forwarded-For*) and the original scheme (*X-Forwarded-Proto: https*), which backends need because TLS ends at nginx.

6.3 B1 - HTTPS with a verified certificate

The decisive HTTPS test was run with verbose output against the domain name and port 8443, **without** *-k*. Certificate verification succeeded, which is the intended outcome of the secure setup.

```

iounu-cliip min/avg/max/sigdev = 12.7/1/107.407/271.000/107.100 ms
mansagarwal@Mansis-MacBook-Pro-6 ~ % curl -v https://app.team1.test:8443/api/status
* Host app.team1.test:8443 was resolved.
* IPv6: (none)
* IPv4: 10.7.21.206
* Trying 10.7.21.206:8443...
* Connected to app.team1.test (10.7.21.206) port 8443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted http/1.1
* Server certificate:
* subject: CN=app.team1.test
* start date: Oct  3 19:47:52 2026 GMT
* expire date: Oct  3 19:47:52 2027 GMT
* subjectAltName: host "app.team1.test" matched cert's "app.team1.test"
* issuer: CN=app.team1.test
* SSL certificate verify ok.
* using HTTP/1.x
> GET /api/status HTTP/1.1
> Host: app.team1.test:8443
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.6
< Date: Sun, 04 Oct 2026 15:03:42 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 29
< Connection: keep-alive
< X-Powered-By: Express
< X-Backend: B
< ETag: W/"1d-eqo46f6cZTMyh8pAIw5ntQG78PA"
<
* Connection #0 to host app.team1.test left intact
{"backend":"B","status":"ok"}
mansagarwal@Mansis-MacBook-Pro-6 ~ %

```

Figure 9. B1 - `curl -v https://app.team1.test:8443/api/status`: TLS 1.3 handshake, certificate verified, HTTP 200 from nginx, X-Backend: B.

Observation in the curl output	Value	Meaning
Name resolution	app.team1.test resolved to 10.7.21.206 (IPv4)	Private DNS answer used, no IP typed by hand
TLS version and cipher	TLSv1.3, AEAD-CHACHA20-POLY1305-SHA256	Modern, encrypted session
Certificate subject	CN=app.team1.test	Name matches the requested host
subjectAltName	host "app.team1.test" matched cert's "app.team1.test"	SAN check passed
Validity period	Oct 3 2026 to Oct 3 2027	Certificate currently valid
Verification result	SSL certificate verify ok	Trusted without -k
Application protocol	ALPN: server accepted http/1.1	HTTP/1.1 inside TLS
HTTP result	200 OK, Server: nginx/1.31.6, X-Backend: B	nginx served and proxied the request

7. Load Balancing (B2)

Six consecutive HTTPS requests were sent to the domain name and only the status line and the *X-Backend* header were kept. The responses alternate perfectly between the two backends, which shows that nginx distributes requests across the upstream group, one request at a time.

```

Last login: Sun Oct  4 19:17:50 on ttys003
mansagarwal@Mansis-MacBook-Pro-6 ~ % for i in {1..6}; do
  curl -s -D - https://app.team1.test:8443/api/status -o /dev/null | grep -E 'HTTP|[X-Backend:]'
done
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
mansagarwal@Mansis-MacBook-Pro-6 ~ %

```

Figure 10. B2 - Six requests to `https://app.team1.test:8443/api/status`: all HTTP 200, X-Backend alternates A, B, A, B, A, B.

Three requests were served by Backend A on Mac 1 and three by Backend B on Mac 2. That a request to one name reaches both machines is the core value of the edge design: capacity and availability are decided at nginx, invisible to the client.

8. Wireshark Network Analysis (C1-C3)

Three captures were taken, one per protocol layer. Together they follow a single request from name lookup through connection establishment to the encrypted exchange.

8.1 C1 - DNS query and response

DNS was captured on the **loopback interface (lo0)** of **Mac 1**, because the client and dnsmasq run on the same machine. The display filter is `dns && dns.qry.name == "app.team1.test"`. Each query (for example frame 25, transaction ID 0xf0fa) is followed immediately by a response (frame 26) carrying **A 10.7.21.206**.

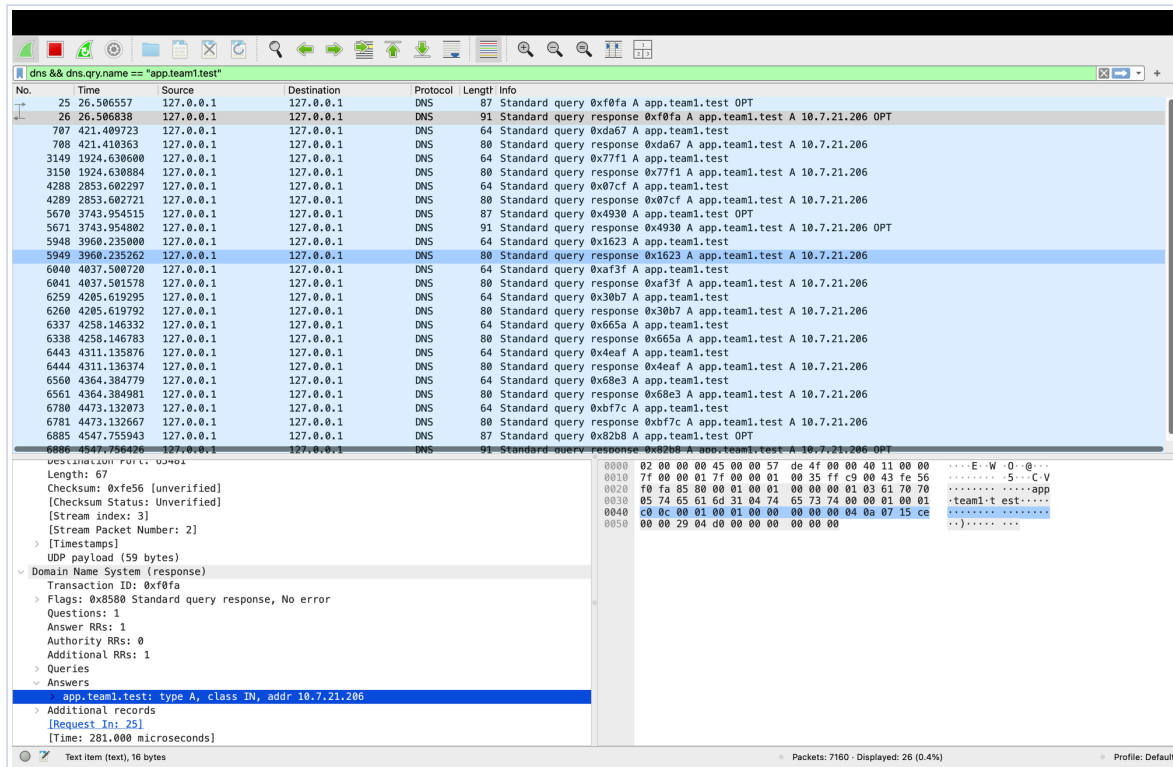


Figure 11. C1 - Wireshark on lo0, filter `dns && dns.qry.name == "app.team1.test"`: query/response pairs; selected response shows the A record 10.7.21.206.

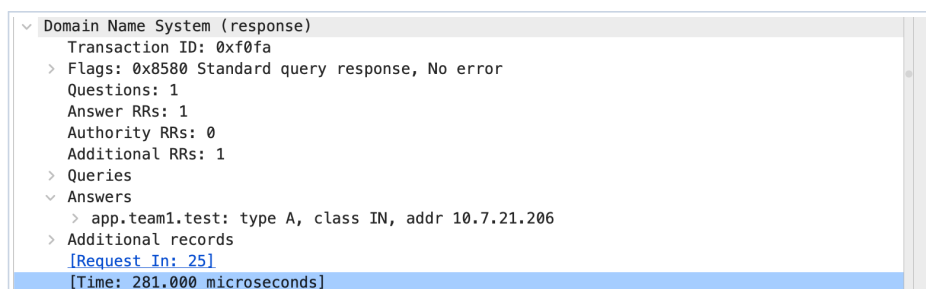


Figure 12. C1 - DNS response detail: flags 0x8580 (standard query response, no error), 1 answer, `app.team1.test` type A class IN addr 10.7.21.206; request in frame 25, answered in 281 microseconds.

The response flags (0x8580) mark a standard response with no error. The 281 microsecond turnaround confirms a purely local exchange. The filter matched 26 packets (13 query/response pairs) in this capture, and every response shown carries the same A record, 10.7.21.206, the nginx edge.

8.2 C2 - TCP three-way handshake

TCP was captured on Mac 2's Wi-Fi interface with the filter `tcp.port == 8443`. The first three packets are the textbook three-way handshake between 10.7.21.182 (client port 53258) and 10.7.21.206:8443 (nginx).

No.	Time	Source	Destination	Protocol	Length	Info
415	-4558.906590	10.7.21.182	10.7.21.206	TCP	78	53258 → 8443 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=3557515030 TSecr=0
438	-4558.855888	10.7.21.206	10.7.21.182	TCP	78	8443 → 53258 [SYN, ACK, ECE] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=64 TSval=4184169760 TSecr=0
441	-4558.787988	10.7.21.182	10.7.21.206	TCP	66	53258 → 8443 [ACK] Seq=1 Ack=1 Win=131776 Len=0 TSval=3557515180 TSecr=4184169760

Figure 13. C2 - The three-way handshake: SYN, SYN-ACK, ACK between 10.7.21.182:53258 and 10.7.21.206:8443.

Frame	Direction	Flags	Meaning
415	10.7.21.182 to 10.7.21.206:8443	SYN, ECE, CWR; Seq=0, MSS=1460, WS=64	Client requests a connection
438	10.7.21.206:8443 to 10.7.21.182	SYN, ACK, ECE; Seq=0, Ack=1, MSS=1460, WS=64	nginx host accepts and acknowledges
441	10.7.21.182 to 10.7.21.206:8443	ACK; Seq=1, Ack=1	Client confirms; connection established
442	client to server	TLS Client Hello (SNI=app.team1.test)	TLS begins immediately on the new connection

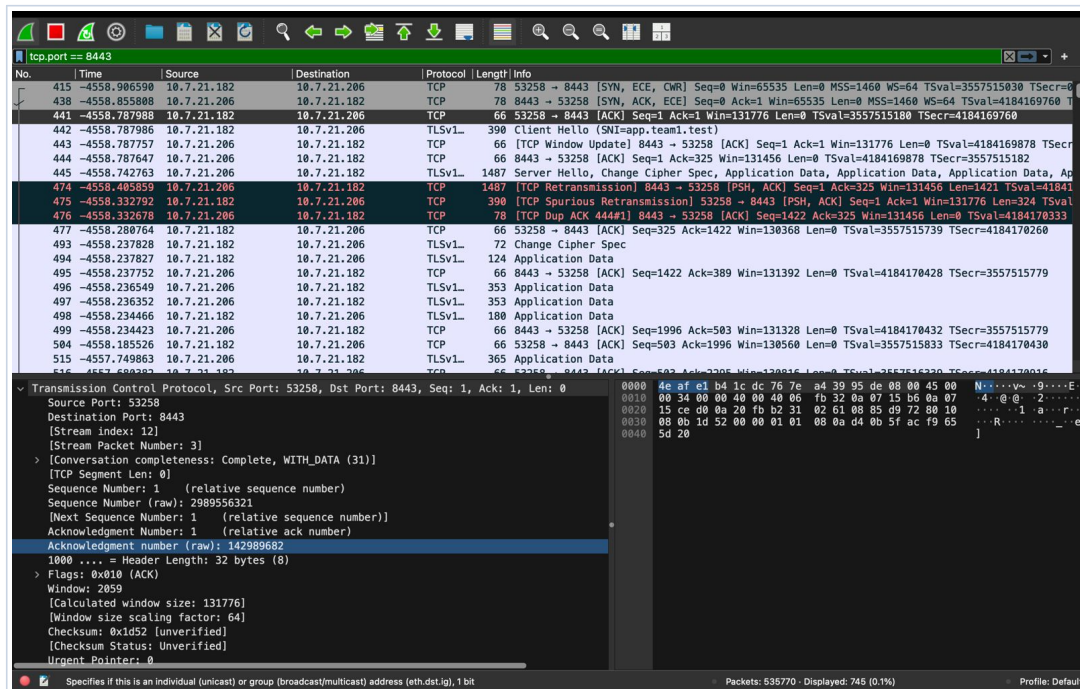


Figure 14. C2 - Full capture view (filter tcp.port == 8443): handshake, Client Hello (SNI=app.team1.test), Server Hello, Change Cipher Spec and Application Data; TCP details of the selected ACK are shown below the list.

The capture also contains a few TCP retransmission and duplicate-ACK rows (frames 474-476) shortly after the Server Hello. These are typical of packet loss on a Wi-Fi link: the 1,487-byte Server Hello segment was resent once, after which the handshake completed normally and encrypted Application Data flowed. They do not affect the result.

8.3 C3 - TLS handshake and certificate inspection

TLS 1.3 encrypts the server certificate, so it cannot be read in a capture of a default curl session. To obtain visible certificate evidence, the handshake was repeated with TLS 1.2 forced:

```
curl --tlsv1.2 --tls-max 1.2 -v https://app.team1.test:8443/api/status
```

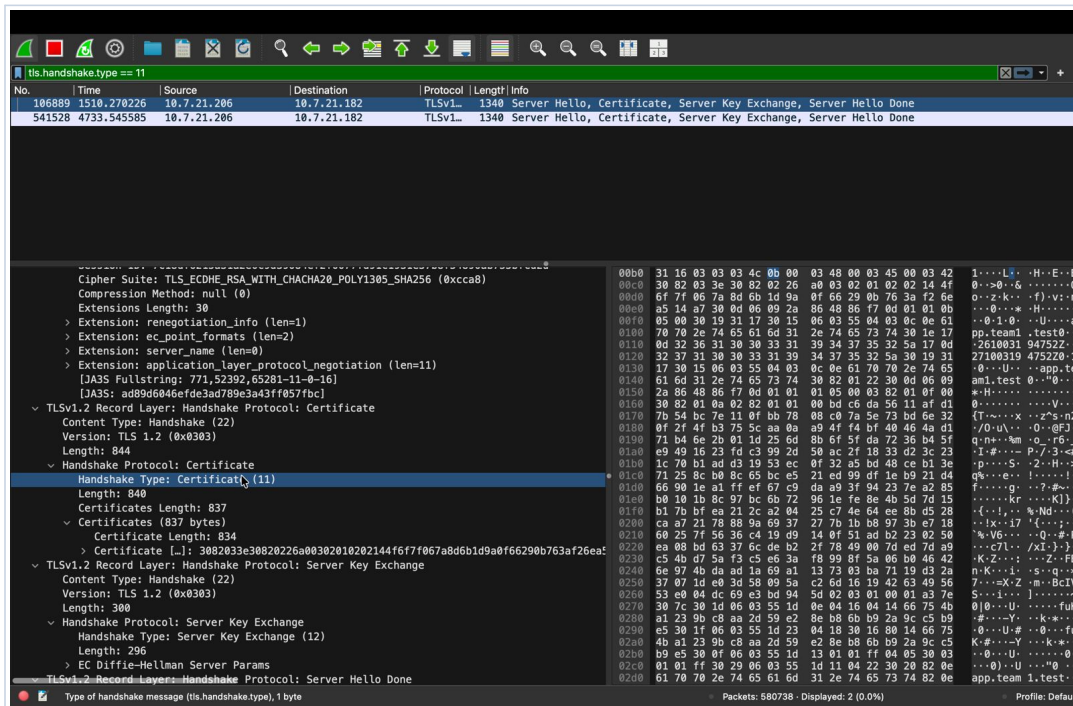


Figure 15. C3 - Filter `tls.handshake.type == 11` (Certificate): the TLS 1.2 server flight (Server Hello, Certificate, Server Key Exchange, Server Hello Done) from 10.7.21.206; the Certificate message (type 11) is expanded, and `app.team1.test` is visible in the raw bytes.

The expanded record shows *Handshake Type: Certificate (11)*, an 840-byte handshake message carrying one 834-byte certificate, and the cipher suite `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8)` with ALPN and `server_name` extensions. The hex pane contains the string `app.team1.test`, confirming that the certificate is issued for the project domain.

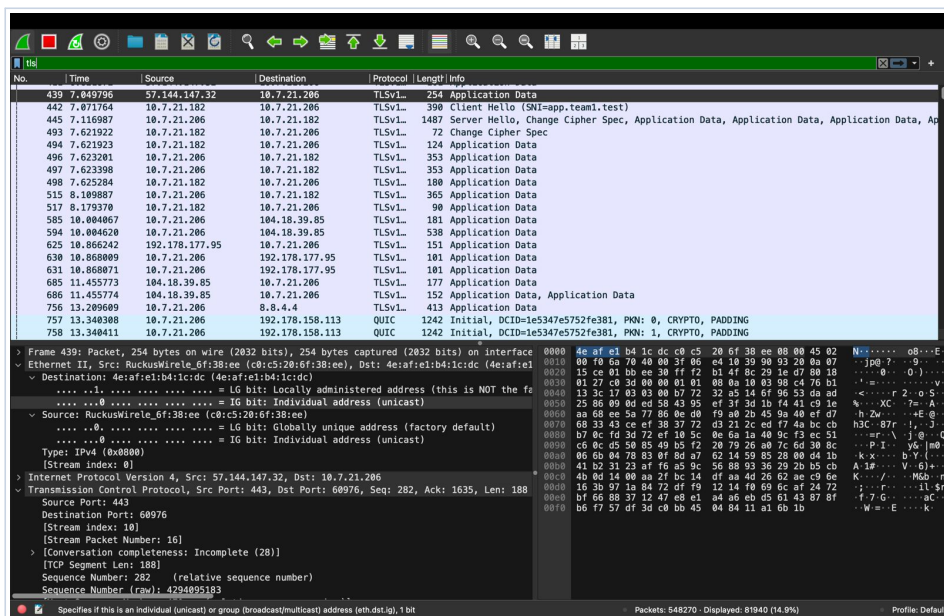


Figure 16. C3 - Filter `tls`: Client Hello (SNI=`app.team1.test`), Server Hello with Change Cipher Spec, then only encrypted Application Data. Other rows are unrelated background TLS/QUIC traffic on the capture interface.

In the default (TLS 1.3) session, the application payload is carried entirely inside *Application Data* records after the Server Hello, so neither the HTTP request nor the JSON response can be read on the wire. This is the observable proof that traffic between client and nginx is encrypted.

Layer	Wireshark evidence	Frames / filter
DNS (UDP 53)	Query and response; A record 10.7.21.206	Frames 25-26; <code>dns && dns.qry.name</code>

Layer	Wireshark evidence	Frames / filter
TCP	SYN, SYN-ACK, ACK to :8443	Frames 415, 438, 441; tcp.port == 8443
TLS handshake	Client Hello (SNI), Server Hello; Certificate (11) in the TLS 1.2 capture	Frames 442, 445 (TLS 1.3); 106889, 541528 (TLS 1.2 Certificate)
Encrypted HTTP	Application Data records only	Frames 494-515; tls

9. HTTP Caching (D1, D2)

9.1 D1 - Cache headers and conditional request

The caching endpoint is `/api/cache`. It was requested through the full HTTPS path (DNS, nginx, backend). The first request returns the full response together with a freshness lifetime and a validator:

```
Backend A running on http://0.0.0.0:3001
mansigarwal@Mansis-MacBook-Pro-6 cn-project % curl -sI https://app.team1.test:8443/api/cache
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sun, 04 Oct 2026 15:07:48 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 77
Connection: keep-alive
X-Powered-By: Express
X-Backend: A
Cache-Control: max-age=60
ETag: "cache-v1"

mansigarwal@Mansis-MacBook-Pro-6 cn-project %
```

Figure 17. D1 - `curl -sI https://app.team1.test:8443/api/cache`: HTTP 200 with Cache-Control: max-age=60 and ETag: "cache-v1" (served by Backend A).

A second request then presents the validator it already holds using *If-None-Match*. Because the resource has not changed, the server answers **304 Not Modified** with headers only and no body:

```
[mansigarwal@Mansis-MacBook-Pro-6 cn-project % curl -i -H 'If-None-Match: "cache-v1"' https://app.team1.test:8443/api/cache
HTTP/1.1 304 Not Modified
Server: nginx/1.31.6
Date: Sun, 04 Oct 2026 15:08:40 GMT
Connection: keep-alive
X-Powered-By: Express
X-Backend: B
Cache-Control: max-age=60
ETag: "cache-v1"

mansigarwal@Mansis-MacBook-Pro-6 cn-project %
```

Figure 18. D1 - `curl -i -H 'If-None-Match: "cache-v1"' https://app.team1.test:8443/api/cache`: HTTP 304 Not Modified, same ETag and Cache-Control.

Notably, the 200 response came from Backend A while the 304 came from Backend B (*X-Backend: B*). Both backends implement the same ETag, so revalidation works regardless of which backend the load balancer picks.

9.2 D2 - Explanation

Header / status	Role in caching
Cache-Control: max-age=60	The response is fresh for 60 seconds . During that time a client or intermediate cache may reuse its stored copy without contacting the server at all.
ETag: "cache-v1"	An opaque validator that identifies this version of the resource.
If-None-Match: "cache-v1"	After max-age expires, the client revalidates by sending back the ETag it holds, asking "has it changed?"
304 Not Modified	The server's answer that the stored copy is still valid. It carries headers but no body , so the unchanged payload is not retransmitted.

In summary, *max-age=60* removes requests entirely while the copy is fresh, and *ETag* with *If-None-Match* turns the requests that remain into cheap header-only exchanges. This saves bandwidth and backend work without risking stale data, because the server can always signal that the resource changed by returning a new ETag and a full 200 response.

10. Failure and Recovery Demonstration (D3)

The failure demonstration used **Option A - stop one backend**. The same six-request loop was run at each stage so that the results are directly comparable. A screen recording covering the sequence accompanies the submission.

Stage	Action	Expected	Observed	Figure
1. Before	Both backends running	A and B both answer	A, B, A, B, A, B (all 200)	19
2. Failure	Backend A stopped on Mac 1	Service continues via B	B, B, B, B, B, B (all 200)	20
3. Evidence	nginx error log inspected	Connection refused to A	connect() failed (61: Connection refused) to 10.7.21.182:3001	21
4. Recovery	Backend A restarted	A rejoins rotation	B, B, A, B, A, B (all 200)	22

```
mansiagarwal@Mansis-MacBook-Pro-6 cn-project % for i in {1..6}; do
  curl -s -D - https://app.team1.test:8443/api/status -o /dev/null | grep -E 'HTTP/|X-Backend:'
done
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
mansiagarwal@Mansis-MacBook-Pro-6 cn-project %
```

Figure 19. D3 stage 1 - Before: both backends healthy; X-Backend alternates A, B, A, B, A, B.


```
mansiagarwal@Mansis-MacBook-Pro-6 cn-project % for i in {1..6}; do
  curl -s -D - https://app.team1.test:8443/api/status -o /dev/null | grep -E 'HTTP|X-Backend:'
done
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
mansiagarwal@Mansis-MacBook-Pro-6 cn-project %
```

Figure 20. D3 stage 2 - After stopping Backend A: all six requests return HTTP 200 with X-Backend: B.

```

}
}
proxy_set_header X-Forwarded-Proto https;
}

tanishas@Tanishas-MacBook-Pro ~ % tail -n 30 /tmp/team1-nginx-error.log
2026/10/04 03:03:42 [error] 5810#0: *61 kevent() reported that connect() failed
GET /api/status HTTP/1.1", upstream: "http://10.7.21.182:3001/api/status", host: "app.team1.test:8443"
2026/10/04 03:03:42 [error] 5810#0: *67 kevent() reported that connect() failed
GET /api/status HTTP/1.1", upstream: "http://10.7.21.182:3001/api/status", host: "app.team1.test:8443"
2026/10/04 19:09:41 [error] 5810#0: *89 kevent() reported that connect() failed
GET /api/status HTTP/1.1", upstream: "http://10.7.21.182:3001/api/status", host: "app.team1.test:8443"
2026/10/04 19:10:39 [error] 5810#0: *95 kevent() reported that connect() failed
GET /api/status HTTP/1.1", upstream: "http://10.7.21.182:3001/api/status", host: "app.team1.test:8443"
2026/10/04 19:28:40 [error] 5810#0: *136 kevent() reported that connect() failed
GET /api/status HTTP/1.1", upstream: "http://10.7.21.182:3001/api/status", host: "app.team1.test:8443"
tanishas@Tanishas-MacBook-Pro ~ %

```

Figure 21. D3 stage 3 - nginx error log (/tmp/team1-nginx-error.log): "connect() failed (61: Connection refused) while connecting to upstream" for http://10.7.21.182:3001/api/status.

```
mansiagarwal@Mansis-MacBook-Pro-6 cn-project % for i in {1..6}; do
  curl -s -D - https://app.team1.test:8443/api/status -o /dev/null | grep -E 'HTTP/|X-Backend:'
done
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
HTTP/1.1 200 OK
X-Backend: A
HTTP/1.1 200 OK
X-Backend: B
mansiagarwal@Mansis-MacBook-Pro-6 cn-project %
```

Figure 22. D3 stage 4 - After restarting Backend A: X-Backend: A reappears in the rotation (B, B, A, B, A, B).

Layer affected. The failure was at the **application/backend layer**. DNS, the network path, the TCP/TLS endpoint and nginx itself all remained available; only the process behind port 3001 on Mac 1 was gone. The error log shows what nginx experienced: the TCP connection to Backend A was actively refused. With its default behaviour, nginx treats a refused connection as a failed attempt and passes the request to the next server in the group, so the client still received HTTP 200 from Backend B.

Recovery. After Backend A was restarted, *X-Backend: A* returned. The first two responses still came from Backend B, which is consistent with nginx's default passive health handling: a server that just failed is skipped for a short period before it is tried again. No configuration change or nginx restart was needed.

11. Testing and Validation Summary

ID	Test	Observed result	Status
T01	Wi-Fi interface identified (A1)	Wi-Fi hardware port is en0	PASS
T02	dnsmasq directives active (A2)	listen-address and both address= records present	PASS
T03	Local DNS on Mac 1 (A3)	NOERROR, A 10.7.21.206 from 127.0.0.1#53	PASS
T04	Cross-machine DNS, Mac 2 to Mac 1 (A3)	Blocked by managed firewall on port 53	LIMITATION
T05	Public DNS negative control (A4)	NXDOMAIN from 8.8.8.8	PASS
T06	Connectivity both ways (A5)	4/4 replies, 0.0% loss each way	PASS
T07	HTTPS, verified certificate (B1)	verify ok without -k; TLS 1.3; HTTP 200	PASS
T08	Load balancing (B2)	A, B, A, B, A, B	PASS
T09	nginx configuration (B3)	Upstream group, listen 8443 ssl, proxy_pass	PASS
T10	DNS capture (C1)	Query/response pairs, A 10.7.21.206	PASS
T11	TCP handshake capture (C2)	SYN, SYN-ACK, ACK on :8443	PASS
T12	TLS handshake capture (C3)	Client Hello (SNI), Certificate (11), encrypted data	PASS
T13	Cache headers (D1)	max-age=60 and ETag "cache-v1"	PASS
T14	Conditional request (D1)	304 Not Modified, no body	PASS
T15	Backend failure (D3)	6/6 HTTP 200 via Backend B	PASS
T16	Recovery (D3)	X-Backend: A reappears after restart	PASS

12. Security Considerations

- **Encryption in transit.** Client-to-nginx traffic is protected by TLS (Figures 9, 15, 16). The nginx-to-backend hop is plain HTTP on the LAN, so that internal segment is not encrypted.
- **Certificate trust.** The certificate is issued for app.team1.test and verified without disabling checks. Because the issuer is the certificate itself, trust exists only on machines that were configured to trust it; it would not be valid for outside users.
- **Private key handling.** The server key stays on the nginx host (path referenced in nginx.conf) and is not published in the repository.
- **DNS trust.** Clients believe whatever the private dnsmasq server answers; plain DNS is unauthenticated in this setup.
- **Backend exposure.** Backends listen on all interfaces, so on this LAN they can be reached without passing through nginx.
- **Evidence hygiene.** Wireshark captures were taken on whole interfaces, so some screenshots contain unrelated background traffic. They are shown as captured.

13. Challenges and Observations

- **Managed-network firewall (A3).** The university network blocks port 53 between client machines. The problem was diagnosed by comparing local resolution (works) with cross-machine resolution (blocked) and is reported as a documented limitation.
- **TLS 1.3 hides the certificate.** Because TLS 1.3 encrypts the Certificate message, a TLS 1.2 handshake was deliberately forced for C3 to make certificate evidence visible. The default session remains TLS 1.3.

- **Non-standard HTTPS port.** Using 8443 means every URL and curl command needs an explicit port.
- **Wi-Fi variability.** Latency ranged from about 13 ms to over 290 ms and a few TCP retransmissions appear in C2; both are normal for shared Wi-Fi and did not affect correctness.
- **Failover semantics.** The D3 error log shows that clients are protected by nginx's retry to the next upstream, but the first requests after recovery still skip Backend A briefly.

14. Limitations

- **A3 (cross-machine DNS) was not achieved** because of the university-managed firewall. Only local resolution on Mac 1 is verified.
- Addresses are DHCP-assigned on Wi-Fi and may change; configuration files would need updating.
- Two backends and six-request tests are small samples; behaviour under concurrency, weights or heavy load was not studied.
- Balancing and failover rely on nginx defaults; no active health checks or explicit retry/timeout directives are configured.
- nginx performs no response caching; caching was demonstrated as server-side validator support (ETag/304).
- Trust is local: the certificate and .test domain work only on machines that were set up to trust them.

15. Conclusion

The project delivers a working private network service platform on two physical Macs and verifies each layer with direct evidence. The private domain *app.team1.test* resolves to the nginx edge and is unknown to public DNS; the edge terminates HTTPS on port 8443 with a certificate that verifies without *-k*; requests alternate between Backend A and Backend B; caching headers and conditional requests behave as specified; and Wireshark shows the DNS exchange, the TCP three-way handshake and the TLS handshake on the wire. When Backend A was stopped, clients continued to receive HTTP 200 responses through Backend B, and Backend A rejoined the rotation after it was restarted.

The one unmet requirement, cross-machine DNS (A3), is attributed to the university-managed firewall and is documented as a limitation rather than hidden. The result is a sound, honestly scoped demonstration of how DNS, TCP, TLS, HTTP caching and reverse-proxy load balancing combine into one service, and of how each can be observed and tested.

Appendix A - nginx Configuration

nginx (from the running nginx.conf, Figure 8)

```

upstream backend_servers {
    server 10.7.21.182:3001;    # Backend A (Mac 1)
    server 10.7.21.206:3002;    # Backend B (Mac 2)
}

server {
    listen 8443 ssl;
    server_name app.team1.test;

    ssl_certificate    <path to certificate>;
    ssl_certificate_key <path to key>;

    location / {
        proxy_pass http://backend_servers;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}

```

The upstream entry for Backend B appears as the Mac 2 LAN address in the captured nginx configuration; the repository README describes the same service as local to Mac 2 (loopback). Both refer to the same process on Mac 2, port 3002.

Appendix B - Verification Commands

```

# DNS
dig @127.0.0.1 app.team1.test          # local, Mac 1
dig @8.8.8.8 app.team1.test           # public negative control

# Connectivity
ping -c 4 10.7.21.206                 # from Mac 1
ping -c 4 10.7.21.182                 # from Mac 2

# HTTPS (never curl -k)
curl -v https://app.team1.test:8443/api/status
curl --tlsv1.2 --tls-max 1.2 -v https://app.team1.test:8443/api/status

# Load balancing
for i in {1..6}; do
    curl -s -D - https://app.team1.test:8443/api/status -o /dev/null | grep -E 'HTTP|X-Backend:'
done

# Caching
curl -sI https://app.team1.test:8443/api/cache
curl -i -H 'If-None-Match: "cache-v1"' https://app.team1.test:8443/api/cache

# Wireshark display filters
dns && dns.qry.name == "app.team1.test" # C1 (lo0, Mac 1)
tcp.port == 8443                        # C2 (Wi-Fi, Mac 2)
tls.handshake.type == 11                 # C3 certificate

```

Appendix C - Evidence Index

Evidence file (evidence/...)	Shows	Figure
A1-dns/image.png	Wi-Fi interface en0	2
A2-domain-resolution/image.png	dnsmasq directives	3
A3-cross-machine-dns/image.png	Local dig on Mac 1 (A3 limitation)	4
A4-public-dns/image.png	NXDOMAIN from 8.8.8.8	5
A5-connectivity/image.png, image copy.png	Ping both directions	6, 7
B3-nginx/image.png	nginx upstream and server block	8
B1-https/image.png	curl -v HTTPS, certificate verified	9
B2-load-balancing/image.png	A/B distribution	10
C1-dns-wireshark/c1.png, c1b.png	DNS query/response	11, 12
C2-tcp-tls/c2b.png, c2a.png	TCP handshake and TLS start	13, 14
C3-tls-details/c3a.png, c3b.png	Certificate (11) and TLS flow	15, 16
D1-caching/image.png, D1-cache-304.png	max-age, ETag, 304	17, 18
D3-failure-recovery/image.png, D3_after.png	Before and after failure	19, 20
D3-failure-recovery/D3-nginx-error.png, D3-restored.png	Error log and recovery	21, 22

Repository structure

```

CN_Project_Elite/
|-- README.md
|-- architecture/architecture.md
|-- config/      dnsmasq.conf, backend-a/ (server.ts, package.json, tsconfig.json)
|-- docs/        D2-caching-explanation.md
`-- evidence/    A1-dns ... A5-connectivity, B1-https ... B3-nginx,
                  C1-dns-wireshark ... C3-tls-details, D1-caching, D3-failure-recovery

```